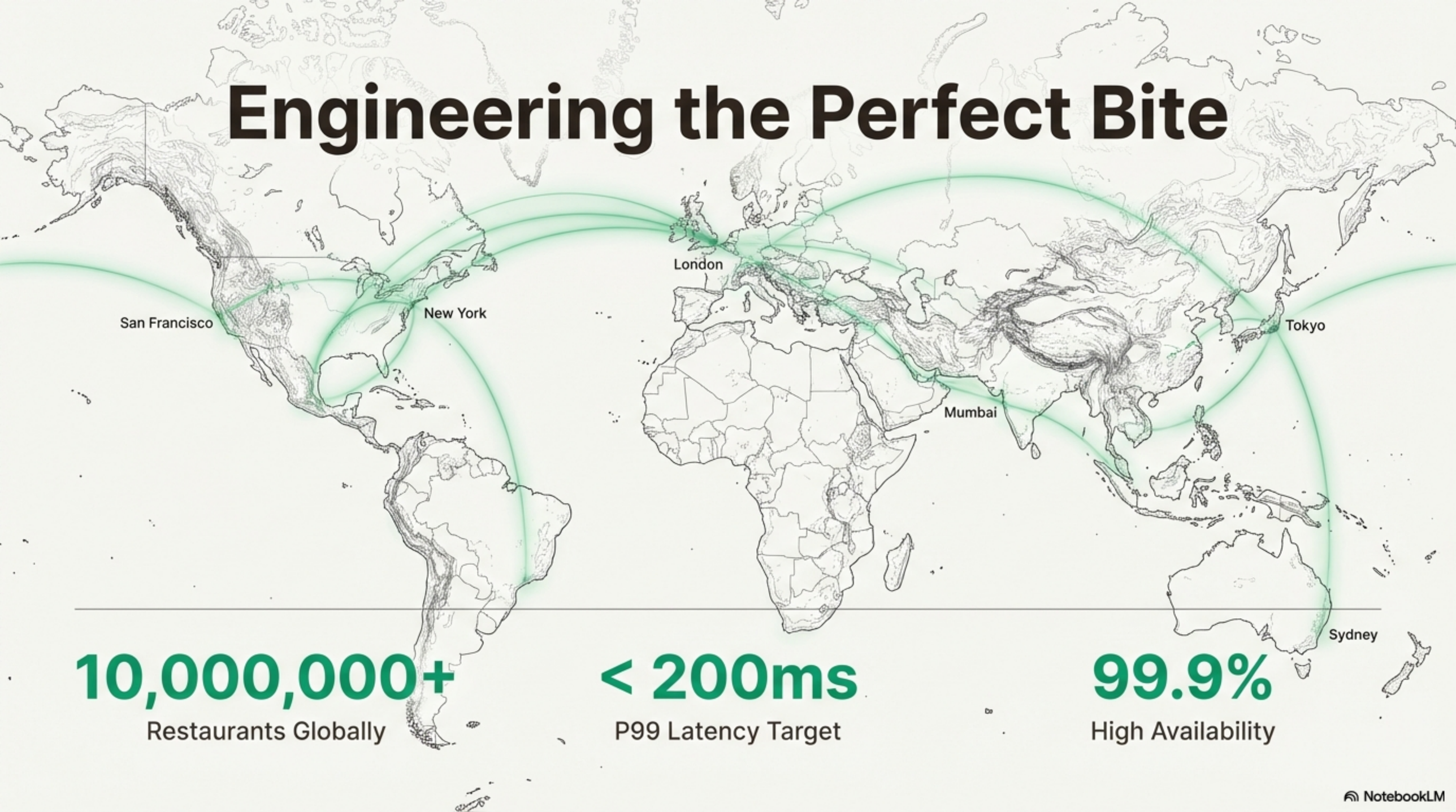
Engineering the Perfect Bite



10,000,000+
Restaurants Globally

< 200ms
P99 Latency Target

99.9%
High Availability

The Challenge: Serving a Hyper-Local, Real-Time World

The Business Goals

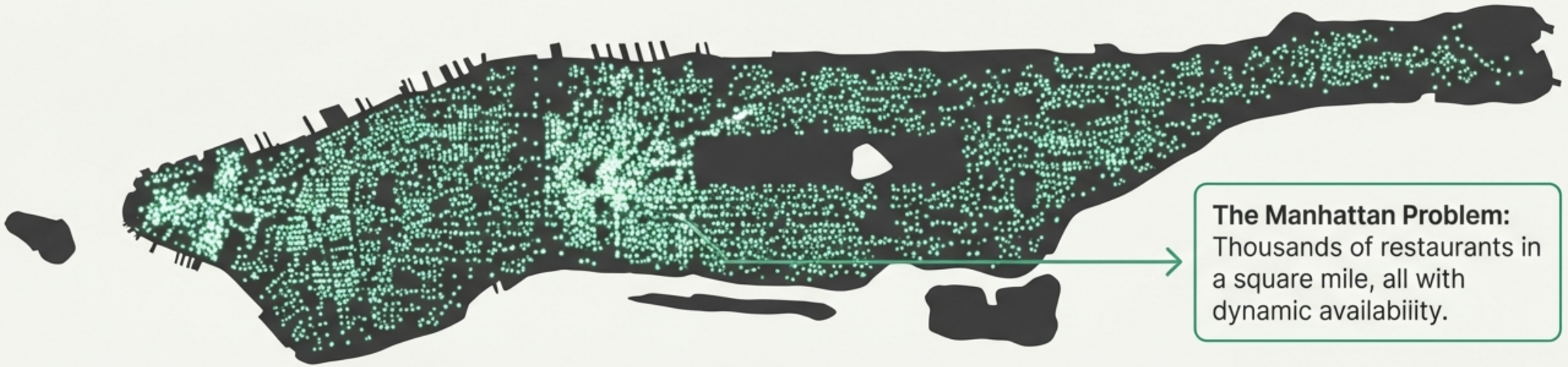
- **Fast Discovery:** Return a ranked feed in < 200ms (P99).
- **Accurate Delivery:** Only show restaurants that can *actually* deliver, right now.
- **Smart Ranking:** Personalize the feed based on relevance signals.
- **Extreme Scale:** Handle 3-5x traffic spikes during meal times.

The Scale of Operation

10,000,000
Restaurants

50,000
Peak Read QPS

200+
Global Cities



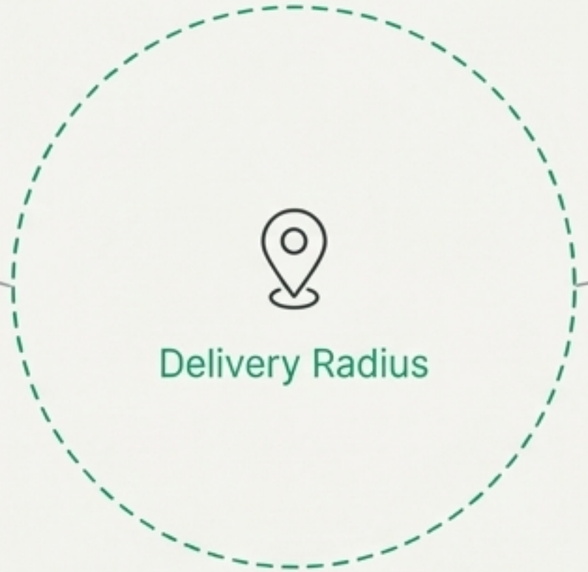
Decision 1: How Do You Search a Planet for a Burrito?

To find **nearby restaurants** in under **50ms**, we need an efficient **spatial index**. The choice defines our system's performance and accuracy.

Geohashing (Rectangular Grid)

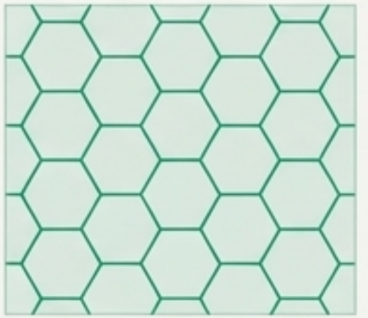
Industry Standard, Prone to Distortion

Delivery Radius



H3 (Hexagonal Grid)

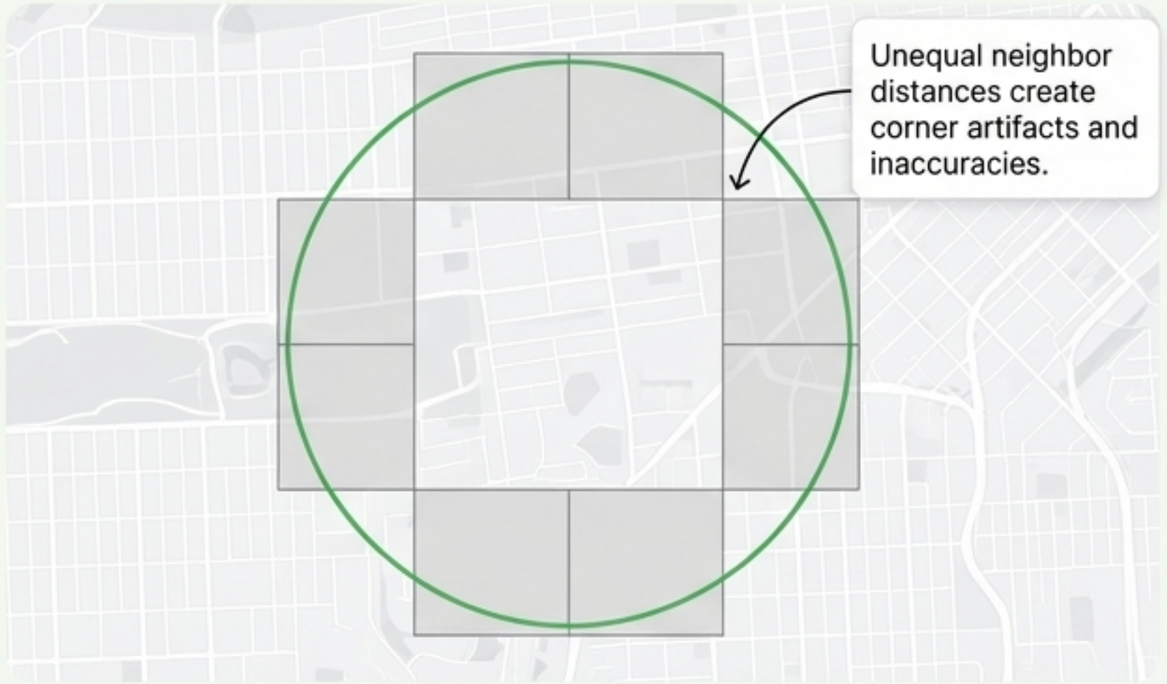
Native Solution, Built for Delivery



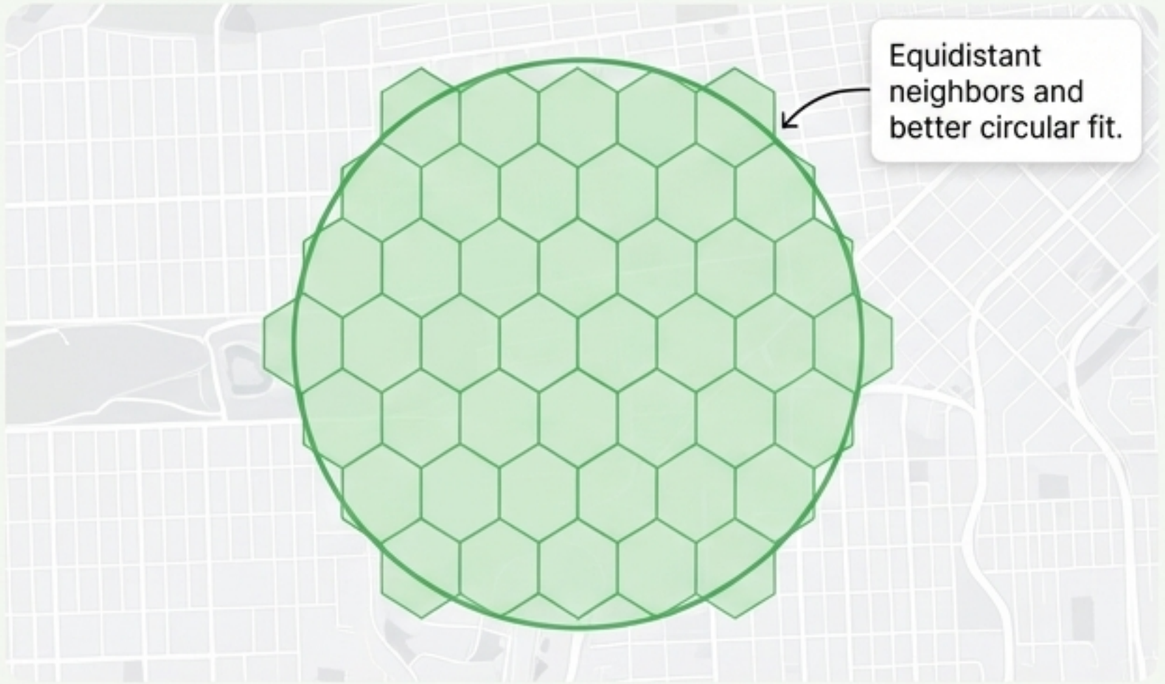
Our primary challenge is geospatial: efficiently finding all restaurants within a delivery radius from a set of 10M+ globally. The index we choose will impact everything from search speed to delivery accuracy.

The H3 Advantage: Why Hexagons Beat Rectangles for Delivery

Geohash



H3



Key Benefits

- **Approximates Circles:** Hexagons fit delivery radii better than squares.
- **Uniform Neighbors:** All 6 neighbors are equidistant, simplifying radius calculations.
- **Native K-Ring Queries:** The killer feature. Finding all cells within a radius is a trivial, built-in operation.
- **Battle-Tested:** Developed and proven at Uber scale for ETAs and dispatch.

> "H3's k-ring function is the key. It allows us to find all restaurants within a K-step radius with a single, highly efficient operation."

Decision 2: How Do You Handle a Constantly Changing City?

A restaurant's availability is not static. It changes second-by-second. Showing a "closed" restaurant is a critical failure.

Visual Story: The "Manhattan Bridge Problem"

A restaurant in Dumbo might stop delivering across the bridge during rush hour. This geo-restriction must be applied *instantly*.

The Architectural Trade-Off

Path A: Index Updates

Change the search index itself.

- ⊕ **Pros:** Zero query-time overhead.
- ⊖ **Cons:** Slow. Propagation takes 1-5 seconds.

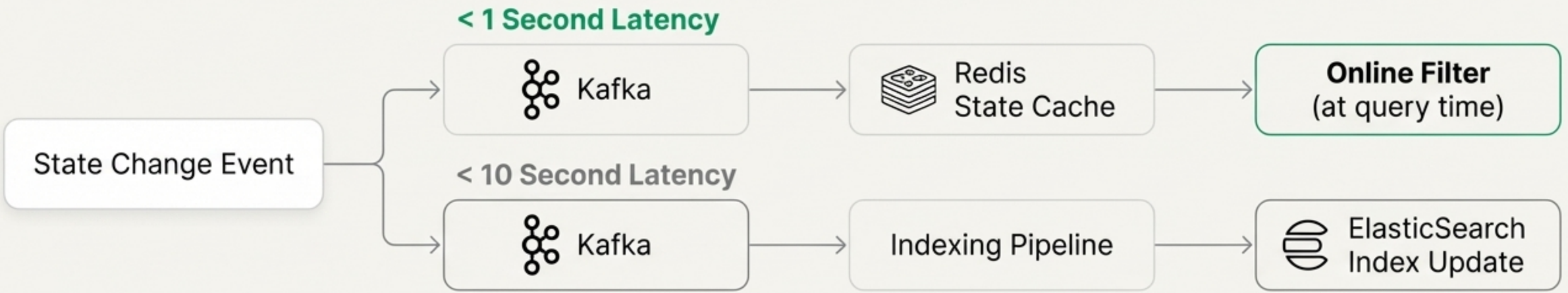
Path B: Online Filtering

Filter results at query time.

- ⊕ **Pros:** Instantaneous changes (<100ms).
- ⊖ **Cons:** Adds 5-10ms overhead to every query.

A Hybrid Reality: Combining Instant Filtering with Indexing

We use a hybrid model, choosing the right strategy based on the urgency of the state change.

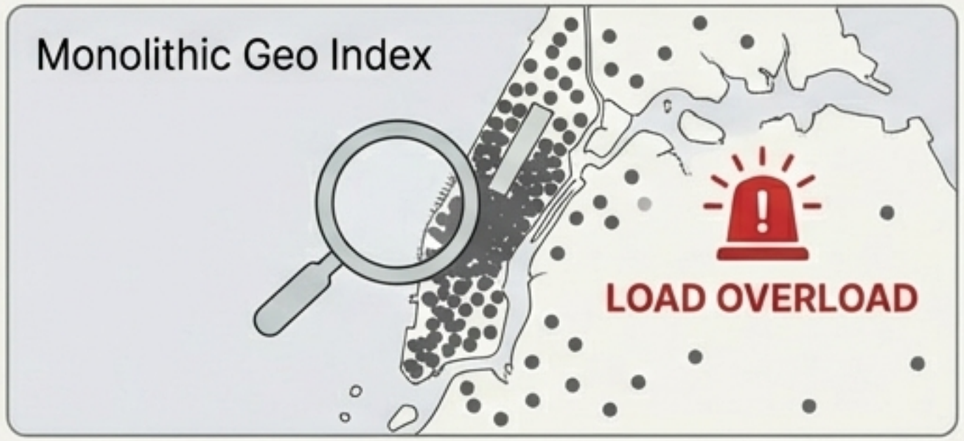


State Change	Target Latency	Strategy Chosen	Justification
Go Offline	< 1 second	Online Filtering (Redis)	Critical to prevent failed orders.
Busy Mode On	< 500ms	Online Filtering (Redis)	Needs to be immediate.
Geo-restriction	< 2 seconds	Hybrid (Redis + ES)	High importance, needs to be fast.
Delivery Zone Change	< 10 seconds	Index Update (ES)	Less frequent, can tolerate slight delay.

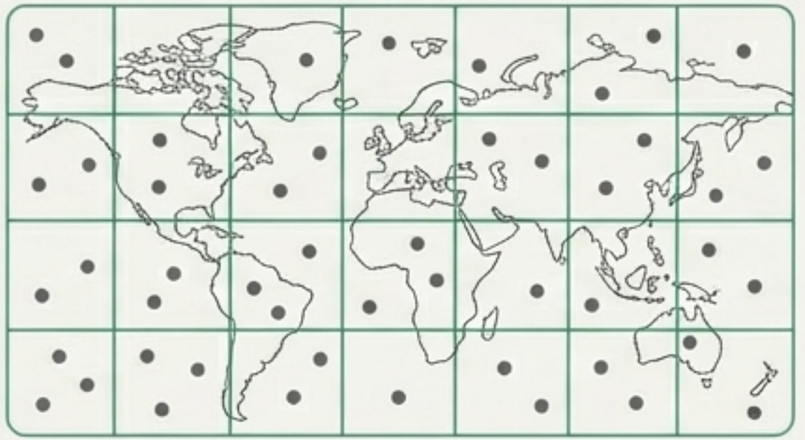
Decision 3: How Do You Scale for Millions of Neighbors?

The Hotspot Challenge: In areas like Manhattan, a single geo-query can involve thousands of restaurants, overwhelming a single database shard. A naive sharding strategy will fail.

The Problem: Monolithic Index



The Solution: Hybrid Sharding



We shard our data in two different ways, for two different jobs.

Our Hybrid Sharding Strategy

For Geo Search

Shard by **Location** (Geohash prefix).

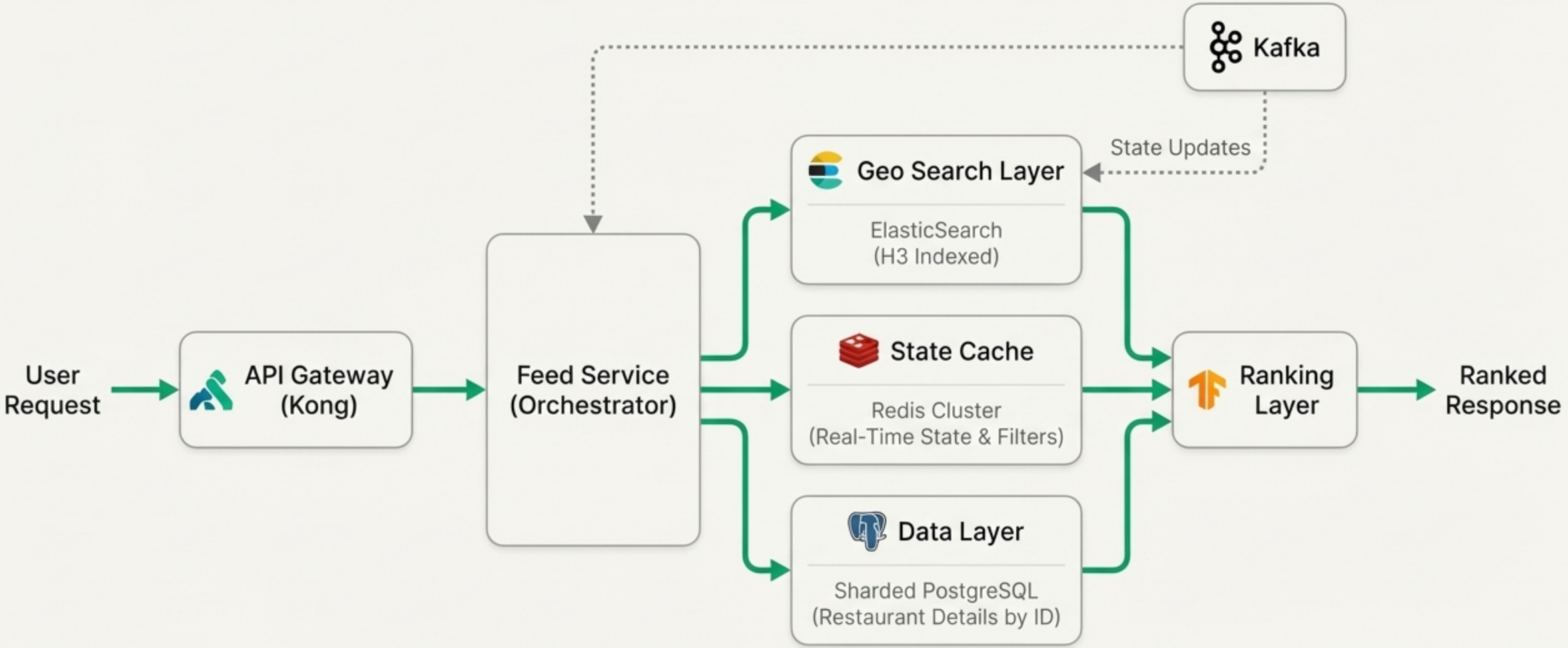
This keeps restaurants that are close to each other on the same shard, optimizing the initial search.

For Restaurant Details

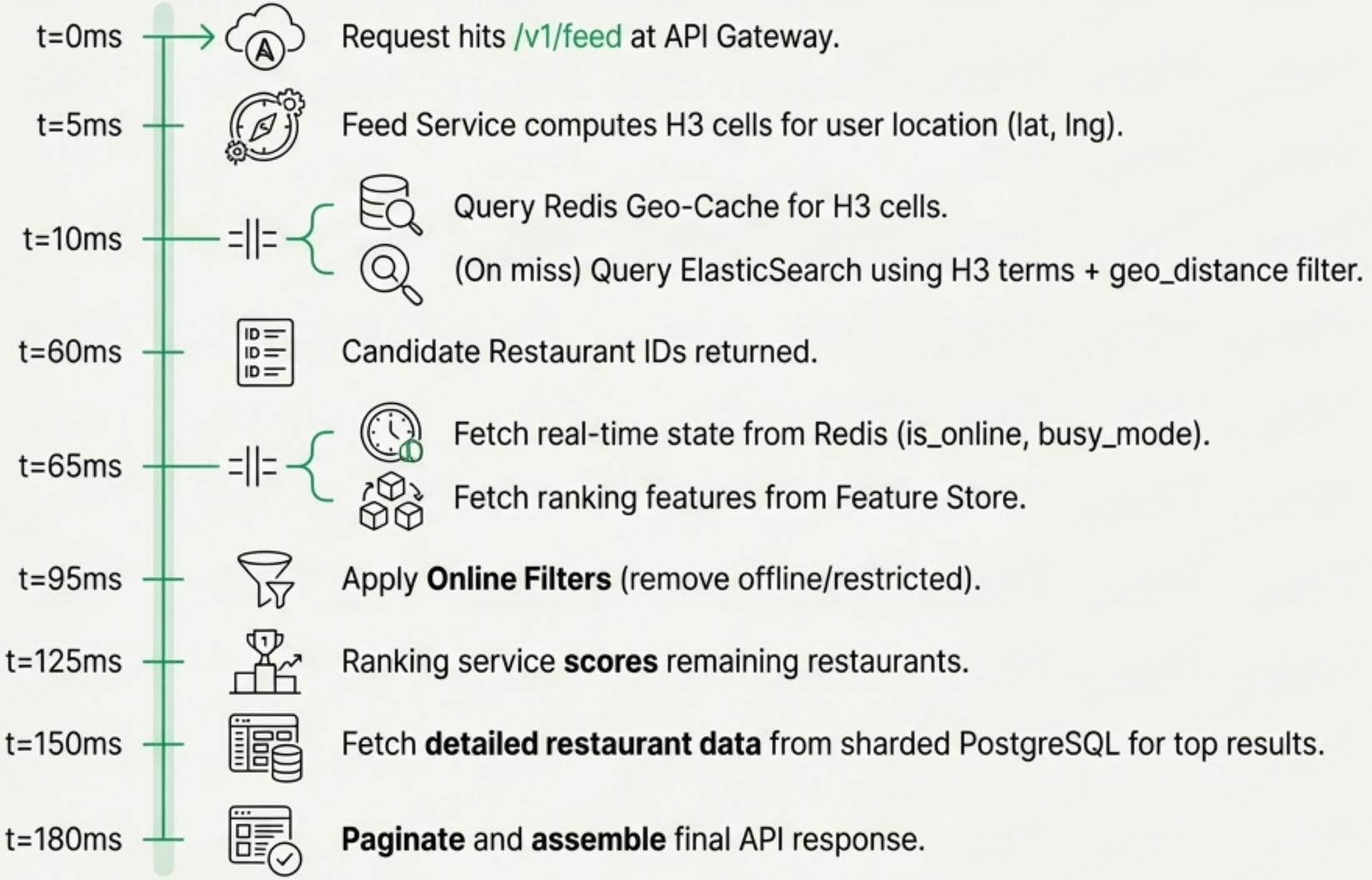
Shard by **Restaurant ID** (Consistent Hashing).

This distributes restaurant data evenly, preventing hotspots when fetching details for a list of restaurants.

The Blueprint: The Uber Eats Feed System Architecture



The <200ms Story: A Request's Journey Through the System



Latency Budget (P99)

Geo Search: **50ms**

State & Feature Fetch: **30ms**

Ranking: **30ms**

Data Hydration & Assembly: **70ms**

Total End-to-End (P99) < 200ms

The Contract: A Clean, Scalable API

Endpoint Focus

`GET` `/v1/feed`

Key Query Parameters

lat, lng (Required)

radius (integer, default: 5000m)

cursor (string for pagination)

cuisine[] (string array for filtering)

sort_by (relevance, distance, rating)

limit (integer, default: 20)

Deep Dive on Pagination

Deep Dive: Why Cursor-Based Pagination?

Problem with Offset: New restaurants added mid-session shift pages, leading to duplicate or skipped results. Deep OFFSET queries are slow in databases.

Our Solution: The cursor encodes the state of the last result. This provides:

1. **Stable Results:** No page-shifting issues.

2. **Performance:** Avoids database OFFSET overhead.

3. **Snapshot Consistency:** Cursor can include a snapshot_id to ensure users paginate through a consistent set of results from their initial query.

The Final Polish: Ranking for Relevance and Discovery

Sort restaurants to maximize user satisfaction and business goals.

Two-Stage Ranking Evolution

V1: Rule-Based Scoring

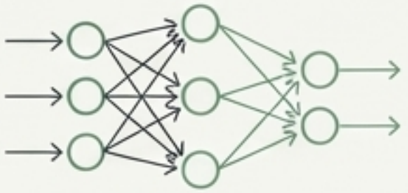
A weighted formula for speed and predictability.

Score = 0.25 DIstance + 0.25 Quality + 0.20 ETA + 0.20 Personalization + 0.10 Business

V2: ML-Based Ranking

A TensorFlow model for nuanced, real-time predictions.

Features: User order history, time of day, cuisine affinities, restaurant popularity.



TensorFlow Model

Measuring Success: A/B Testing Metrics

Metric	Definition	Target
CTR	Clicks / Impressions	> 15%
Conversion	Orders / Clicks	> 25%
Restaurant Diversity	Unique cuisines in top 20	> 8
New Restaurant Exposure	Impressions for new partners	> 5%

Built for Failure: Resilience by Design

In a distributed system, failures are inevitable. The architecture must gracefully handle outages of its dependencies.

Failure: ElasticSearch Cluster Down

Impact: No new geo searches.

Mitigation: Serve stale results from Redis Geo-Cache; enable a circuit breaker to prevent cascading failures.

Failure: Redis Cache Down

Impact: High latency; massive load shifts to ElasticSearch & Postgres.

Mitigation: Degraded mode. Bypass cache with strict rate limiting on direct DB/ES queries.

Failure: ML Ranking Service Unavailable

Impact: No personalized ranking.

Mitigation: Fallback instantly to the V1 rule-based scoring (distance + rating).

Failure: Full Regional Outage

Impact: Users in one region are down.

Mitigation: Automated DNS failover to the nearest healthy region. RTO: < 5 minutes.

The Scorecard: A System that Delivers

Speed

< 200ms

P99 End-to-End Latency

Fast, fluid discovery for a seamless user experience.

Scale

50,000

Peak Queries Per Second

Effortlessly handles global lunch and dinner rushes.

Accuracy

< 1 sec

Critical State Propagation

Drastically reduces failed orders by showing real-time availability.

Reliability

99.9%

Uptime for Read Operations

A highly available system that eaters and restaurants can trust.